

Week 4 • Replacing local storage with a real database

Technical Interaction Design · ITU · Autumn 2026

Built 23 September 2026 · commit f7bf164

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-04.pdf>

CONTENTS

Backends and Low-Code Backends

- What a backend is
- Low-code backends and Backends-as-a-service
- Parse as a low-code backend

Replacing `localStorage` with a Real Database

- Connecting the app to a backend
- Creating and Saving An Object to the Database
- Reading Objects from the Database
- Putting the snippets into the app
- Creating: in the handler that adds a to-do
- Reading: once, when the page opens
- Updating: only the field that changed
- Deleting: the same handle, destroyed
- The four of them, in one file
- What we have just written has a name: CRUD
- Code organization
- What came up in the lecture

The Three States of Remote Data

- The synchronous case hid the pattern
- A fetch has three states, and your component has to render all three
- `failed` is not the optional one
- The fourth state, and the bug it explains
- What you render in each state is a design decision
- You are going to write this again, and again

Debugging

- `debugger;` is a real statement, and costs nothing when nobody is watching
- React DevTools lets you see state instead of printing it
- Your editor can pause code that is running in the browser
- The stack frame you want is the topmost one in your own file

Backends and Low-Code Backends

Source: [Lectures/Technical/Backend/Backends-and-Low-Code-Backends.md](#) — something unclear? [Open an issue](#), or send a pull request.

Motivation: we want to be full-stack web developers :) But we don't have much time.

Read before the lecture: [Async Programming and Promises](#). Everything you ask a backend for arrives later, which in JavaScript means a **promise**. From the first `save()` onwards this lecture assumes you have met one, and knows the difference between `.then()`, `.catch()` and `await`.

What a backend is

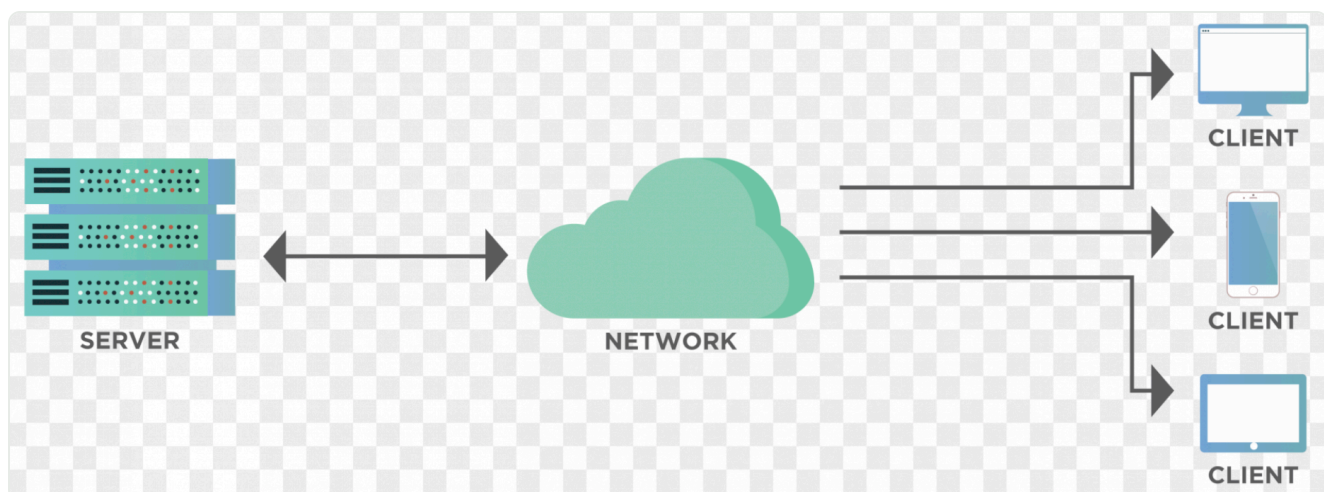
“Backend” only means something relative to a front-end

- **Front-end** – code that runs in the user's browser and handles presentation and user interaction
- **Back-end** – handles data processing, storage, and security

Client-server is the arrangement, and the backend is the server half

The pair has a name, and it is older than the web: a *client-server architecture*.

The **client** runs on the user's machine and asks for things; the **server** runs somewhere else and answers.



The detail that matters is the one the picture shows and the phrase does not: **there are many clients and one server**. Nearly everything in the next three weeks follows from that asymmetry.

- **The server is the only shared thing**, so it is the only place where two users can meet. Sharing a to-do list is possible because there is one copy of it and both clients are talking to that copy.

- **The server is the only trusted thing.** Every client runs on somebody else's computer, in code they can read and change. Anything you need to be *true has to be enforced on the server* — which is next week's lecture in one sentence.
- **The server is far away.** Between the asking and the answering there is a network, and your interface has to have something to show meanwhile.

Note: Client / front-end are going to be used interchangeably in this course.

Backend Responsibilities: The backend takes care of everything the browser cannot be trusted with

1. Authentication (proving that a user is who they say they are)
2. Authorization (checking what a user is allowed to do)
3. Session management (tracking a user across requests, so they don't log in again on every click — this is a result of HTTP being a stateless protocol)
4. Business logic and DB access (because it's the one central place, as per above)
5. Scheduled jobs (e.g., `cron`, backups, etc.)
6. API endpoints / request handling (since the backend receives and responds to requests)
7. Data validation (ensuring incoming data is correct/safe)

Infrastructure: A traditional backend needs that you configure many pieces of infrastructure before writing even a line of your own code

1. **Machine** setup (or create a VM with a cloud provider)
2. **Operating system** installation & configuration
3. Security & **firewall** configuration
4. **Database** management system (DBMS)
5. **Web server** — application that listens to HTTP requests and serves pages to clients (e.g. nginx, apache2)
6. **Application server** — runtime environment in which your application will be executed
7. **Logging, monitoring & analytics** — super essential for a service that has to be available non-stop
8. **Backup** system

Note that none of the eight is on the previous list. That list was about *responsibilities*; this one is about the *machinery* you would have to assemble before you could discharge a single one of them.

Something unclear in this section? Open an issue.

Low-code backends and Backends-as-a-service

A low-code backend hands you (some of) the eight responsibilities already built

The responsibilities arrive already assembled, and what you get is a **pre-built solution** for the needs that every application has, so that the only code you write is the code particular to your application domain.

Low-code says how much you have to write to implement your backend.

Backend-as-a-service (BaaS) gives you a low-code backend with the infrastructure also pre-built

As-a-service says that it is somebody else that owns the infrastructure. All you are responsible with is writing the application.

The two are independent and can be combined

- **Parse Platform is a low-code backend.**
 - Open-source software
 - You rent a virtual machine in the cloud and install it there, or you run it on your own laptop while you are developing
 - You pay a fixed price for renting the VM
 - Or you pay electricity for the machine in your basement / data center
- **Firebase is a backend-as-a-service.**
 - Proprietary, hosted by Google
 - Renting it is the only way to have it - you pay for the usage of the firebase backend
 - However, open source alternatives exist
- **Back4App is backend-as-a-service** that comes with Parse pre-installed on it.
 - Somebody else running the low-code backend you could have run yourself
 - You don't have to rent a VM
 - You just pay for the usage of the backend

The code you write that communicates with a Parse server, does not care whether the server is hosted on your own VM or is a service hosted by somebody else

The only thing that your code needs to know is that there's a Parse backend. And what's the URL where it can communicate with that backend.

It will be the same with any kind of backend. The frontend talks to a given IP address or URL.

That backend can be running on your local machine, then you connect to `localhost:1337` e.g., or `127.0.0.1:1337`. Or you can connect to a staging server. Or to the main server.

Something unclear in this section? Open an issue.

Parse as a low-code backend

Is open source

Now. It started as a startup, got bought by Facebook, and is now opened again

The Parse server is implemented in Node

Implemented in JS. And as we've discussed in the past, you can run JS in the browser, or on any machine with the help of the `node` Javascript interpreter that does not need a browser.

Parse has an answer for every line of the requirements of a backend

The backend is responsible with ...	Parse gives you
Authentication	<code>Parse.User</code> — signup, login, sessions (<i>next week</i>)
Authorization	class-level permissions and ACLs (<i>next week</i>)
Session management	completely free (handled by the SDK, and remembered across page reloads)
Business logic and DB access	the JavaScript SDK — the subject of today
Scheduled jobs	cloud jobs
API endpoints	REST and GraphQL, generated from your classes (although not needed when you use the SDK)
Data validation	cloud code triggers (<i>week 11</i>)

And two more that were never on the list, because the browser never had them to lose:

- **file storage**, and
- an interactive **dashboard** for looking at and editing your database

For your frontend/client code, Parse provides you with an easy to use Javascript SDK

SDK

- stands for (software development kit)
- is a library that makes it easy for you to communicate from the frontend to the backend
- without this SDK, you would have to communicate with the backend with the HTTP protocol, and that's more clumsy

Everything you do to Parse goes through the JavaScript SDK. There are also SDKs for other languages than JS (Kotlin for Android, Swift for iOS, etc.).

Note: The full documentation of the SDK is in the [Parse.js Javascript Guide](#). Use it as reference – that is, search for things as you need them. Don't read it as a book.

Everything above is what a backend *is*. What it looks like to actually put one under an app you have already written is the next note: [Replacing localStorage with a Real Database](#).

Something unclear in this section? Open an issue.

Exam Questions

1. What is the difference between front-end and back-end?
2. List at least 5 responsibilities of a backend.
3. What is Parse Platform and what does it provide?

Replacing localStorage with a Real Database

Source: [Lectures/Technical/Backend/Backends-Low-Code-Backends-and-the-Parse-Platform.md](#) — something unclear? [Open an issue](#), or send a pull request.

The to-do list works, and everything it knows dies with the browser that stored it. This note replaces `localStorage` with a real database — Parse, on Back4App — one operation at a time, in the app we already have. It assumes [Backends and Low-Code Backends](#), and it assumes you have met a promise: from the first `save()` onwards, everything you ask the backend for arrives later.

Connecting the app to a backend

Three steps, and none of them mentions a to-do: an app on Back4App, which hands you the keys; the SDK in the project; and the connection configured with those keys. At the end of them the app can reach a database, and has not yet put anything in it.

Creating the backend for our app on Back4App

1. Create an account on Back4App
2. Create a app for your react application in Back4App (this is the backend of your application)
3. Somewhere in settings find `APP_ID` and `JAVASCRIPT_KEY` and `PARSE_SERVER_URL` and save them for later

Installing the Javascript SDK so we can use it from React

```
npm install parse@8.6.0 events
npm list parse
```

Check what the second command prints. It should say `parse@8.6.0` — the version we use in the course. Asking for the version by name is what keeps everyone on the same one: with a bare `npm install parse`, `npm` is free to hand you an older release instead.

Your Node version. Run `node --version`: anything from **20.19 onwards** is fine. That floor comes from Vite rather than Parse — on an older Node `npm run dev` will not start at all, so you find out immediately. Parse itself declares support only for the long-term-support releases, so on Node 25 `npm` prints an `EBADENGINE` warning when you install it. It installs and works anyway; the warning is advisory and you can ignore it.

`parse` is the SDK. `events` is a library that Parse needs in order to run in the browser; without it, `Parse.initialize()` fails with `Emitter is not a constructor`.

That failure has a shape you will meet again: `npm run build` **succeeds**, with at most a warning in the output. The app only dies when someone loads it. A green build is not evidence that anything

works.

Initializing the connection to the backend

We put the following lines as early in the application as possible, e.g., in `App.jsx`.

```
import Parse from 'parse';

Parse.serverURL = "https://parseapi.back4app.com/"; // your PARSE_SERVER_URL
Parse.initialize("YOUR_APP_ID", "YOUR_JAVASCRIPT_KEY");
```

The initialization configures your react application to connect to - the server (`Parse.serverURL`) - the corresponding app (`YOUR_APP_ID`), because there might be multiple apps on the server

Something unclear in this section? [Open an issue](#).

Creating and Saving An Object to the Database

```
import Parse from 'parse';

// This next line is super advanced - it creates a Class!
// How we do know, look on the next line, we create an object of that class there!
const TodoItem = Parse.Object.extend("TodoItem");
const newItem = new TodoItem();

newItem.set("text", "Call the landlord");
newItem.set("done", false);

newItem.save()
  .then(onSuccessfulSave)
  .catch(onError);

function onSuccessfulSave(savedItem) {
  alert("saved a todo with id: " + savedItem.id);
}

function onError(error) {
  alert(error.message);
}
```

Steps:

1. `Parse.Object.extend("TodoItem")` creates a class for the object

2. `save()` - sends the data to the server
3. `save()` returns a *promise*: `.then()` runs if it worked, `.catch()` runs if it did not

This code does not float somewhere in the component: it goes **in the event handler** — the function that already runs when the user submits the form. Adding a to-do is something the user *does*, and the save is part of doing it.

The callback is called `onSuccessfulSave` rather than `onSave` on purpose: `save` in the app means the button the user pressed, and `save()` here means the round trip to the database.

`onSuccessfulSave` also says the thing that matters about it — it runs only if the database said yes.

Classes created with `Parse.Object.extend` correspond to tables in the database

Thus in JS you create an object of that class and when you save it it gets automatically saved in the database.

If a class created with `Parse.Object.extend` does not exist in the database, it is automatically created

The `TodoItem` class is automatically created in the database if it didn't exist. This behavior can be turned off, and we *will* turn off next week. You will see why.

The promise can be written also with anonymous functions

The two functions are given names on purpose. Written inline it is the same code, and that is the form you will meet often, especially in the examples, tutorials, and AI-generated code.

```
newItem.save()
  .then((savedItem) => alert("saved a todo with id: " + savedItem.id))
  .catch((error) => alert(error.message));
```

Names make it easier to read. Compare the above once more with the code below:

```
newItem.save()
  .then(onSuccessfulSave)
  .catch(onError);

function onSuccessfulSave(savedItem) {
  alert("saved a todo with id: " + savedItem.id);
}

function onError(error) {
  alert(error.message);
}
```

Reading Objects from the Database

The simplest type of query

- is created **for a specific class**
- given some **some constraints**
- then **executed with `find()`**

```
const TodoItem = Parse.Object.extend("TodoItem");

// we are creating a query object for objects of type TodoItem
const query = new Parse.Query(TodoItem);

query.equalTo("done", false);
query.ascending("createdAt");

// await
const results = await query.find();

for (const item of results) {
  console.log(item.id + " - " + item.get("text"));
}
```

`find()` returns a promise as well. From here on we use `await` rather than `.then()`, because the code then reads in the order in which it happens.

`await` and `.then()` are the same promise, handled two ways. `.then(f)` says *when the answer arrives, call `f` with it* — and the lines after it run immediately, before `f` ever does. `await` says *stop here until the answer arrives, then carry on with it in hand* — so the next line is the next thing that happens.

Nothing changes on the network; what changes is whether your code reads top to bottom or is cut into callbacks. The price of `await` is that it is only allowed inside a function marked `async`, which is why `async` starts appearing on our handlers below.

References:

- [Query Constraints](#)
- [Queries on Strings](#)

Something unclear in this section? Open an issue.

Putting the snippets into the app

You already have a working app. It has a list, a form, a checkbox and a delete button, and at the bottom of `ToDoList.jsx` it has this:

```
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, [todos]);
```

One line, and everything is saved. Whenever anything about any to-do changes, the entire array is serialized and written again.

The whole-array write is the first thing that has to go

With `localStorage` that line is free. It is the same machine, the list is a few kilobytes, and nobody notices.

Now put a network in the middle. Ticking one checkbox would upload every to-do you own. Two people editing the same list would overwrite each other wholesale, because each of them is writing the entire list as they last saw it. And there is no operation in the database for *replace everything*; there are operations for one object at a time.

So the single effect that wrote everything becomes **one call per change**:

User action	In the database
adds a to-do	create one object
ticks a checkbox	save one object
presses Delete	destroy one object
opens the page	find all the objects, once

This is not a Parse rule. It is what having a backend means.

We now go through that table one row at a time. All four end up in the same file, `ToDoList.jsx`, in the handlers that are already there.

Something unclear in this section? [Open an issue](#).

Creating: in the handler that adds a to-do

```
const TodoItem = Parse.Object.extend("TodoItem");

async function handleAdd(newTask) {
  const item = new TodoItem();
  item.set("text", newTask);
  item.set("done", false);
  const saved = await item.save();

  setTodos([...todos, { id: saved.id, text: saved.get("text"), done: saved.get("done") }]);
}
```

Two things happen, and the order matters: **first the database, then the state**. If the save fails, the to-do never appears on the screen — which is the truth.

What `save()` gives back is a Parse object, and the component has been working with plain `{ id, text, done }` all along, so we unpack it on the way into the state.

Note: that `id` is the one thing that is not a field: it lives on the object itself, not behind `get()`.

That `id` also retires something. Until now each to-do got its React key from `crypto.randomUUID()`, a value invented in the browser purely to stop React warning about a list without keys. The database has a better one. `objectId` is unique because the database guarantees uniqueness, it is the same value on every device that loads the list, and it survives a refresh, which a generated key never did. From here on the key in the JSX is the row's real identity instead of a number we made up.

Something unclear in this section? [Open an issue](#).

Reading: once, when the page opens

Reading already happened once, when the component mounted — but it happened *while* it mounted:

```
let [todos, setTodos] = useState(loadTodos); // reads localStorage, synchronously
```

`localStorage` could answer inside that line. A database cannot: `find()` hands back a promise, and `useState` needs a value now. So the read moves out of the initial state and into an effect that runs once, after the first render:

```
useEffect(() => {

  async function load() {
    const query = new Parse.Query(TodoItem);
    query.ascending("createdAt");
    const results = await query.find();

    setTodos(results.map((each) => ({
      id: each.id,
      text: each.get("text"),
      done: each.get("done"),
    })));
  }

  load();

}, []);
```

The empty `[]` is essential: it says *run this once, when the component first appears*. Without it the effect would run after every change to `todos` — and since it ends by setting `todos`, each load would trigger the next one.

Notice what the state now starts as: `useState([])`, an empty list. Not because the list is empty, but because we do not know yet. We will come back to that.

Why not `useEffect(async () => ...)`

Because React reads whatever an effect returns as its **cleanup function** — the thing to call when the component goes away. An `async` function always returns a promise, so React would be handed a promise where it expects a function.

So the asynchronous work goes into a *separate* function; the effect calls it and returns nothing. Where you declare that function is then up to you. Above, it sits inside the effect, which is what most codebases that fetch will show you: it is used nowhere else, so it lives where it is used.

The other option is to declare it in the component body:

```
async function loadTodos() {  
  // ... the same body  
}  
  
useEffect(() => {  
  loadTodos();  
}, []);
```

which is what `todo-26` does, and what the room preferred when we put both on screen. It reads as a function and a call to it — a plainer shape than declaring a function and invoking it on the next line, and one you can copy out on its own. The cost is that the loading logic no longer sits beside the `[]` that says when it runs.

Either is correct. The only rule is that **the effect itself must not be `async`**.

A footnote if you run the linter: with the function declared outside, `react-hooks/set-state-in-effect` warns about `setTodos`. The warning is wrong — that `setTodos` is behind an `await`, so it is not synchronous — but the rule cannot see through the declaration to find out. Inside the effect it stays quiet; outside, it is noise you should recognise rather than obey.

Something unclear in this section? Open an issue.

Updating: only the field that changed

Ticking a checkbox means changing one field of one row. We do **not** need to download the row first:

```
async function handleToggle(id) {  
  
  const todo = todos.find((each) => each.id === id);  
  
  const item = TodoItem.createWithoutData(id);  
  item.set("done", !todo.done);  
  await item.save();  
  
  setTodos(todos.map((each) => (each.id === id ? { ...each, done: !each.done } : each)));  
}
```

The order is the same as in the create, and here the reason is sharper. Suppose the state went first: the checkbox ticks immediately, then the save fails — server down, network gone — and the screen now shows a to-do as done while the database says it is not. The app has told the user something untrue, and they will find out the next time they open it somewhere else.

You can watch this happen. Turn the network off in the browser's devtools and tick a checkbox: `save()` rejects, the line below it never runs, and the checkbox stays where it was. Nothing changes on screen, which is the truth.

`createWithoutData(id)` deserves the objection: **why don't we look the object up first?**

The obvious version is this, and it works:

```
const item = await new Parse.Query(TodoItem).get(id); // ask the server for the to-do
item.set("done", done);
await item.save(); // ask it again, to change one field
```

Two round trips, and the first one downloads a to-do we already have on screen only to throw it away.

A Parse object is not a *copy of a row* but rather a *handle to a row*.

The intuition tells us that a Parse object is a *copy of a row*. It is not. It is a handle to a row. A reference to it.

An object is a **handle to a row** — and a handle needs only the id. `createWithoutData(id)` builds one out of thin air, and `save()` sends only the fields you actually changed, so what crosses the network is `{done: true}` and an id.

The handle has one blind spot, worth knowing before the group project. Because we never read the row, we never learn what it says *now*: `!todo.done` is computed from our own copy, which was accurate when the page loaded. If somebody else has ticked the same to-do since — your flatmate, on their phone, against the same database — you are flipping a value that is already stale. With one person and one list this never bites. With two people writing to the same row, the round trip you saved is the price of being right, and `new Parse.Query(TodoItem).get(id)` is how you pay it.

Something unclear in this section? Open an issue.

Deleting: the same handle, destroyed

To destroy a row you need to know *which* row, and nothing else:

```
async function handleDelete(idToDelete) {
  const item = TodoItem.createWithoutData(idToDelete);
  await item.destroy();

  setTodos(todos.filter((each) => each.id !== idToDelete));
}
```

Something unclear in this section? Open an issue.

The four of them, in one file

```
import { useState, useEffect } from "react";
import Parse from "parse";

const TodoItem = Parse.Object.extend("TodoItem");

export default function ToDoList({ firstName }) {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    async function load() {
      const query = new Parse.Query(TodoItem);
      query.ascending("createdAt");
      const results = await query.find();

      setTodos(results.map((each) => ({
        id: each.id,
        text: each.get("text"),
        done: each.get("done"),
      })));
    }
    load();
  }, []);

  async function handleAdd(newTask) {
    const item = new TodoItem();
    item.set("text", newTask);
    item.set("done", false);
    const saved = await item.save();

    setTodos([...todos, { id: saved.id, text: saved.get("text"), done: saved.get("done")}]);
  }

  async function handleToggle(id) {
    const todo = todos.find((each) => each.id === id);

    const item = TodoItem.createWithoutData(id);
    item.set("done", !todo.done);
    await item.save();

    setTodos(todos.map((each) => (each.id === id ? { ...each, done: !each.done } : each)));
  }
}
```

```

    async function handleDelete(idToDelete) {
      const item = TodoItem.createWithoutData(idToDelete);
      await item.destroy();

      setTodos(todos.filter((each) => each.id !== idToDelete));
    }

    // ... the JSX is unchanged
  }

```

The `useEffect` that used to synchronize the app with `localStorage` is gone, because there is nothing left for it to synchronize. Every handler does the same two things in the same order: **change the database, then change the state.**

The app now works against a real database, and two people opening it see the same list.

Something unclear in this section? [Open an issue.](#)

What we have just written has a name: CRUD

Four operations, and between them they are most of what any application does to its data:

CRUD	The user...	In Parse
Create	adds a to-do	<code>new TodoItem()</code> , <code>set()</code> , <code>save()</code>
Read	opens the page	<code>new Parse.Query(TodoItem)</code> , <code>find()</code>
Update	ticks a checkbox	<code>createWithoutData(id)</code> , <code>set()</code> , <code>save()</code>
Delete	presses Delete	<code>createWithoutData(id)</code> , <code>destroy()</code>

You will meet the acronym in documentation, in job ads, and in the exam. It is worth noticing how little there is to it: you have now written all four, and the rest of the course is mostly about doing them *safely*, and against [more than one class at a time](#) (both next week), and *efficiently* (the week after).

Something unclear in this section? [Open an issue.](#)

Code organization

Look at the one file above once more. `TodoItem` is set up there, the three-line unpacking of a Parse object into `{ id, text, done }` appears twice, and the component that draws a list of to-dos is also the component that knows a field is called `"text"`. It works, and it is already getting hard to read — and this is one class and four operations.

Rendering the UI and talking to the database are two different jobs

So, instead of calling Parse directly from our UI components, we put those calls in their own file — a **service layer**.

Without one, a component both renders the UI *and* talks to the database, which are two responsibilities (see the **Single Responsibility Principle**, in the further reading). Components get longer, and the same four lines of Parse setup get copied into every one of them:

```
// in the component - messy, and about to be duplicated in the next component  
const TodoItem = Parse.Object.extend("TodoItem");  
const item = new TodoItem();  
item.set("text", text);  
item.set("done", false);  
await item.save();
```

With a service layer the component says what it wants:

```
// in the component - clean, simple  
await createTodo(text);
```

Only one file should know that a field is called `text`

Our services folder starts with a single file, `src/services/todoService.js`:

```
import Parse from "parse";

const TodoItem = Parse.Object.extend("TodoItem");

// React is happier with plain JS objects than with Parse objects,
// and this is also where we unify the treatment of `id` with the other fields
function toPlainObject(parseObject) {
  return {
    id: parseObject.id,
    text: parseObject.get("text"),
    done: parseObject.get("done"),
  };
}

export async function fetchTodos() {
  const query = new Parse.Query(TodoItem);
  query.ascending("createdAt"); // oldest first
  const results = await query.find();
  return results.map(toPlainObject);
}

export async function createTodo(text) {
  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);
  return toPlainObject(await item.save());
}

export async function setTodoDone(id, done) {
  const item = TodoItem.createWithoutData(id);
  item.set("done", done);
  return toPlainObject(await item.save());
}

export async function deleteTodo(id) {
  const item = TodoItem.createWithoutData(id);
  await item.destroy();
}
```

`toPlainObject` is the only place in the whole app that knows a field is called `"text"`. Everywhere else, a to-do is `{ id, text, done }` — the same shape it had when it lived in `localStorage`, which

is why the components barely change.

`createWithoutData(id)` is the same handle-to-a-row trick we used in the handlers above: `setTodoDone` and `deleteTodo` never read the row they are about to change.

The component keeps its shape, and loses every mention of Parse

The component keeps exactly the shape it had a page ago. What changes is that no line of it mentions Parse any more — each handler says *what it wants*, and the service says how:

```
import { useState, useEffect } from "react";
import { fetchTodos, createTodo, setTodoDone, deleteTodo } from "../services/todoService";

export default function ToDoList({ firstName }) {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    async function load() {
      setTodos(await fetchTodos());
    }
    load();
  }, []);

  async function handleAdd(newTask) {
    const created = await createTodo(newTask);
    setTodos([...todos, created]);
  }

  async function handleDelete(idToDelete) {
    await deleteTodo(idToDelete);
    setTodos(todos.filter((each) => each.id !== idToDelete));
  }

  async function handleToggle(id) {
    const todo = todos.find((t) => t.id === id);
    await setTodoDone(id, !todo.done);
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }

  // ... the JSX is unchanged
}
```

Same two things in the same order as before — **change the database, then change the state** — but a handler is now three lines you can read at a glance, and `"text"`, `"done"` and `TodoItem` appear in exactly one file in the whole app.

The version above is not finished: it assumes the data arrives. What a component has to show while it has not arrived — and when it never does — is [The Three States of Remote Data](#), which every component you write from here on will need.

Something unclear in this section? [Open an issue](#).

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned.

`Parse.Object.extend` is not how you declare a class

Asked in class what that line actually is, given that it is plainly not JavaScript class syntax. It is not. It is closer to a small language of its own, built out of function calls, which produces a class *at runtime*. You can tell it really is a class, because `new TodoItem()` works and `new` only works on classes.

The same magic maps it onto a table you never created. That is the deal Parse offers, and it is worth naming it as a deal: a great deal of typing disappears, and in exchange you do not know exactly how your class became a table. Most of the time that is fine. It stops being fine on the day the mapping does something you did not expect.

A word about the exam

Said in the room, and repeated here for whoever was not: `useState` and `useEffect` are the floor. There is nothing more basic in React than those two. `useEffect(..., [])` meaning *run once, when the component first appears* is the kind of thing you will be asked to explain, and not being able to is the sort of gap that fails an exam by itself. If the empty array is still mysterious, go back to [The useEffect Hook](#) before next week.

“API” turned out to mean four different things

`save()` prompted the question of what an API actually is, and answering it took a quarter of the lecture, because the word is doing at least four jobs at once: the **browser API** (`history.back()`, speech recognition — the functions and classes the browser hands your code), an **SDK** like the Parse one you are using, an **HTTP API** whose operations are URLs you send requests to, and a **REST API**, a strict subset of that with firm rules about naming — rules most projects claim to follow and quietly do not.

The part that belongs to this note: your `item.save()` becomes an HTTP request. The app id and the JavaScript key travel with *every* call, because HTTP is stateless and the server remembers nothing about you between requests; the object goes up as JSON and comes back as JSON. The SDK exists so that you never write that by hand.

The keys are written in plain text in every request your app sends

Open your own to-do app, open the developer tools on the **Network** tab, and tick a checkbox. One request appears. Click it, and look at what your app sent:

```
POST https://parseapi.back4app.com/parse/classes/ToDoItem/AbC123xY

{"done":true, "_method":"PUT",
 "_ApplicationId":"YOUR_APP_ID",
 "_JavaScriptKey":"YOUR_JS_KEY"}
```

There they are. Not hidden in the bundle, not obfuscated: **in the body of every single request**, put there by your own code. `_ApplicationId` and `_JavaScriptKey` say **which application is calling**. Every visitor has them. They identify your app, and they prove nothing about the person using it.

Anybody who can open your app can read them. That is not a bug in Parse, and there is no setting that fixes it: a key that the browser must send is a key the browser's owner can read. The request also shows the **class name** (`ToDoItem`) and the **object id**, so a stranger now knows what your tables are called, and can address individual rows in them. What to do about that is the subject of [next week](#).

We went into it far enough in the room for now; the rest, including calling the same server with `curl` and `fetch` instead of the SDK, is in [Web Service APIs](#), which comes back later in the course next to efficient communication with the backend.

Something unclear in this section? Open an issue.

Exam Questions

1. What does CRUD stand for and what do each of the letters represent?

2. Explain what this code does:

```
const ToDoItem = Parse.Object.extend("ToDoItem");
const newItem = new ToDoItem();
newItem.set("text", "Buy milk");
newItem.set("done", false);
await newItem.save();
```

3. Once the to-dos live on a server, this one line has to become several separate calls. For each thing the user can do in the app, name the database operation that replaces it.

```
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, [todos]);
```

4. What are two benefits of placing this code in a separate service file rather than directly in a React component?

```
export async function fetchTodos() {  
  const query = new Parse.Query(TodoItem);  
  query.ascending("createdAt");  
  const results = await query.find();  
  return results.map(toPlainObject);  
}
```

5. How would you query all TodoItems where done is false, ordered by creation date?

6. Your `loadTodos` queries the database, builds an array of to-dos, and ends with `return todos`. It is called from a `useEffect`, and nothing appears on screen. Why does returning the data achieve nothing here, and what must the function end with instead?

7. A query returns the right number of rows, but every to-do renders blank. What is wrong, and why does `id` keep working when `text` does not?

References

The documentation on ParsePlatform.org

- [Getting Started Guide](#) - extensive reference for everything ParseJS

The Three States of Remote Data

Source: [Lectures/Technical/React/The-Three-States-of-Remote-Data.md](#) — something unclear? [Open an issue](#), or send a pull request.

Up to now, every component that had data simply had it. This note is about what happens to a component when its data lives somewhere else — on the far side of a network, fifty milliseconds away on a good day and several seconds from a bad train connection — and the answer is not “it waits”, because components cannot wait. It renders anyway, and you have to decide what it renders.

The synchronous case hid the pattern

Here is how the to-do list loaded its data when the data was in `localStorage` :

```
let [todos, setTodos] = useState(loadTodos);
```

A `useState` initializer has to **return the value**. `loadTodos` could, because `localStorage.getItem` is synchronous: the data is already on the machine, so the line after the call has it.

Reading the data and having the data happen at the same instant. There is no in-between moment, so there is nothing to render during it, so the component has exactly one state: the data.

This is a special case, and it is the only one you have met so far. It stops being available the moment the data is on a server. `fetchTodos()` cannot be a `useState` initializer, because it does not return to-dos — it returns a promise, and React would put the promise in your state.

Something unclear in this section? [Open an issue](#).

A fetch has three states, and your component has to render all three

The initial state can now only be “nothing yet”, and the data has to arrive later, which is why it arrives through an effect. And the arrival itself has to become state, because the component needs to render differently before and after it:

```
function ToDoList() {
  const [todos, setTodos] = useState([]);
  const [status, setStatus] = useState("loading");

  useEffect(() => {
    async function load() {
      try {
        setTodos(await fetchTodos());
        setStatus("ready");
      } catch (error) {
        console.error(error);
        setStatus("failed");
      }
    }
    load();
  }, []);

  if (status === "loading") return <p>Loading your to-dos...</p>;
  if (status === "failed") return <p>Could not reach the server. Try again.</p>;

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

Loading, failed, ready. Three screens, from one fetch.

The rendering itself is nothing new — it is the early-return form of [conditional rendering](#), which is why that note showed you three shapes and said the `if` was the one for whole-screen decisions. This is the screen it meant.

Something unclear in this section? [Open an issue](#).

failed is not the optional one

It is the one people leave out, because during development the server is on your own machine and it never fails.

Without it, a dead network looks exactly like an empty list. The user retries nothing, because as far as they can see nothing went wrong. A `catch` that only writes to the console is a bug with a comforting shape: you see the error while developing, and your users never do.

Something unclear in this section? [Open an issue.](#)

The fourth state, and the bug it explains

A `ready` list can still have nothing in it, and “*you have no to-dos*” is a real screen that a real app needs. It is often the nicest screen in the app — the one that tells a new user what to do first.

So the states are really:

state	what it means	what the user should see
loading	I have not heard back yet	a skeleton, or a quiet message
failed	I asked, and it went wrong	what went wrong, and a way to retry
empty	I heard back: there is nothing	an invitation to add the first one
ready	I heard back: here it is	the data

And now the bug in the to-do list has a name. The app already had an empty state:

```
{todos.length === 0 ? <Nothing to do> : ( ... )}
```

When the data moved to a server, no loading state was added — so **the empty state impersonated the loading state**. `useState([])` gives an empty array, `length === 0` is true, and the screen cheerfully says *Nothing to do* when the truth is *I do not know yet*. Those are different things, and the user cannot tell them apart.

This is the general shape of the mistake: when a state is missing, some other state takes its place, and it is usually the one that looks most like success.

Something unclear in this section? [Open an issue.](#)

What you render in each state is a design decision

`<p>Loading...</p>` is honest and takes a minute. A spinner is a rotating element in CSS, and there is nothing React-specific about it. Better than either, when you know the shape of what is coming, are **skeleton rows** — grey blocks the size of the real to-dos — because the layout does not jump when the data lands.

A spinner has a failure mode of its own: if the data usually arrives in 200ms, it appears and disappears before the eye resolves it, and the flicker reads as a glitch rather than as progress. Showing nothing for the first few hundred milliseconds often looks *faster* than showing a spinner immediately.

And for `failed`, the word “Retry” on a button is worth more than any amount of apology in the message. An error the user can do something about is a smaller error.

Something unclear in this section? [Open an issue.](#)

You are going to write this again, and again

Every screen that fetches needs these states, which means the same pair of `useState` calls in every component that talks to the backend. Duplication that shows up in every component of a certain kind is usually telling you something, and later in the course we come back and do something about this one.

Something unclear in this section? [Open an issue.](#)

Exam Questions

1. Why could `useState(loadTodos)` load the to-dos from `localStorage`, but cannot load them from a backend?
2. Name the three states a fetching component has, and say what the user sees in each.
3. This component fetches in a `useEffect` and starts with `useState([])`. What does the user see before the data arrives, and why is it wrong?
4. What is the difference between the `empty` state and the `loading` state, and what goes wrong when a component has only one of them?
5. Why is it not enough to `console.error` in the `catch`?

Debugging

Source: [Lectures/Technical/React/Debugging.md](#) — something unclear? [Open an issue](#), or send a pull request.

Three tools, in the order you will reach for them: a `debugger;` statement, the React DevTools component tree, and a proper breakpoint in your editor. Then a word on reading the error before you touch anything.

`debugger;` is a real statement, and costs nothing when nobody is watching

`debugger;` is a real JavaScript statement, not a library call. Put it anywhere in your code: it does nothing while nobody is watching, and pauses execution the moment DevTools is open.

That makes it the cheapest breakpoint there is — and unlike a `console.log`, you cannot forget where you left it, because it stops the program and shows you.

Something unclear in this section? [Open an issue](#).

React DevTools lets you see state instead of printing it

[React Developer Tools](#) is a browser extension — install it once, for Chrome, Firefox or Edge, and it adds two panels that plain DevTools does not have:

- **Components** — the live component tree, with every component's props and state visible and editable. This is the only way to see state rather than print it, and it is what turns "the list did not update" into "the state changed but the [key](#) did not", which are different bugs.
- **Profiler** — which components re-rendered and why. Ignore this until something is slow; it answers a question you do not have yet.

Something unclear in this section? [Open an issue](#).

Your editor can pause code that is running in the browser

In VS Code, this takes a one-off configuration.

1. Create the `.vscode` folder in your project
2. Add the following configuration in the `launch.json` file

```
{
  "configurations": [
    {
      "type": "chrome",
      "request": "launch",
      "name": "Debug localhost in Chrome ",
      "url": "http://localhost:5173/",
      "webRoot": "${workspaceFolder}/src"
    }
  ]
}
```

3. Set your breakpoints

4. Select from the menu: `Run → Start Debugging`

Something unclear in this section? [Open an issue.](#)

The stack frame you want is the topmost one in your own file

Most React errors name the component and the hook. Read the first three lines of the stack before changing anything: the frame you want is usually the topmost one that belongs to *your* file rather than to `react-dom`.

Something unclear in this section? [Open an issue.](#)

Exam Questions

1. What does `debugger;` do when nobody has DevTools open, and why does that make it safer to leave in than a `console.log`?

2. “The list did not update.” Name two different bugs that look exactly like that on screen, and say how the Components panel tells them apart.

3. An error’s stack is fifteen frames deep and most of them are inside `react-dom`. Which frame do you read first, and why that one?